KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

R E S U L T R E P O R T

마이크로프로세서응용및실험 결과보고서_ADC



수 강 과 목	: 마이크로프로세서응용및실험-2분반
담 당 교 수	: 김도인 교수님
학 번	: 202210820
작 성 자	: 이동제
공 동 실 험 자	: 강민수(202214221)
제 출 일	: 2024-06-13 목요일

1. 실험목적

ATmega128에 내장된 ADC(Analog-to-Digital Converter)에 대해 배운다.

2. 실험원리

ADC 기능을 통해 아날로그 신호를 디지털 값으로 변환하는 과정을 실험합니다.

ADC란 Analog-to-Digital Converter로 실제 물리량으로 존재하며 연속적인 값을 갖는 아날로그 신호를 컴퓨터가 알 수 있는 형태로 이산화된 값인 디지털 값으로 변화시키는 기능을 뜻합니다.

제작한 ATmega128 마이크로컨트롤러 보드의 ADC는 다음과 같은 특징을 가지고 있습니다.

Analog-to-Digital Converter의 특징
1. 아날로그 신호를 10비트 디지털 값으로 변환할 수 있다. 최대 1,024개의 서로 다른 값으로 아날로그 신호를 표현할 수 있다. (=10비트 분해능 : 0~5V 범위를 2^10개(1024)로 쪼개어서 표현 가능한 단위를 말한다.)
2. 변환기에서 일어날 수 있는 오차의 최대 범위가 0.5 LSB 이내로 유지된다. 높은 선형성을 통해 변환 결과의 정확성을 보장한다.
3. 실제 측정값과 디지털 값 간의 최대 오차가 ±2 LSB를 넘지 않는다.
4. 아날로그를 디지털 값으로 변환하는 시간은 13μs~260μs 사이에서 소요된다.
5. 초당 최대 76,900개(76.9 kSPS)의 샘플을 처리할 수 있으며, 최대 해상도에서는 초당 최대 15,000개(15 kSPS)의 샘플을 처리할 수 있다.
6. 8개의 서로 다른 단극성 입력(아날로그 신호) 채널을 선택하여 변환할 수 있다.
7. 7쌍의 차동 입력 채널이 사용될 수 있으며, 두 개의 핀에 입력되는 아날로그 입력 신호 간의 차이를 측정하는 것을 뜻한다.
8. 10배과 200배 증폭. 두 개의 차동 입력 채널 중 선택할 수 있다.
9. ADC 변환 결과를 읽을 때, 값을 좌측으로 조정하여 읽을 수 있는 옵션이 있다.
10. 0V(Ground)에서 5V(VCC)까지의 아날로그 입력 전압을 지원한다.
11. 기준 전압을 2.56V로 선택할 수 있다. (ADC 변환의 기준이 되는 전압)
12. 연속적으로 변환을 수행하는 모드와 한 번 변환시킨 후 멈추는 모드. 두 가지 모드를 지원한다.
13. 슬립 모드에서 노이즈를 줄여주는 기능을 제공한다.

여기서 8채널 아날로그 입력을 포트F로 입력받아 멀티플렉서에 의해 A/D 변환기로 연결됩니다.

우리는 ATmega128의 61번 핀인 PF0/ADC0만 가변저항에 연결되었기 때문에, 다른 핀과 전위차를 측정할 수 없으므로 차동

입력 대신 단극성 입력으로 PF0을 통해서만 실험을 진행합니다.

가변저항에 드라이버를 조절하며 실제 아날로그 전압을 조절할 수 있습니다.

그리고 AVcc라고 디지털 전원 핀인 Vcc와는 별개로 아날로그 전원 핀인 AVcc가 따로 존재합니다.

A/D 변환기 클럭은 변환되는 주기를 뜻하며, ATmega128의 기본 클럭 주파수인 16MHz를 분주비로 나누어서 만드는 값입니다.

그리고 변환 시간마다 사이클이 다른데 최초 변환 시 25s, 최초 변환 이후 단극성 입력은 13초로 최초 변환은 좀 오래 걸려도 그 후부터는 짧아집니다.

단극성 입력의 A/D 변환 식은 $\frac{Vin \times 1024}{V_{REF}}$ 로 V_{IN}는 단극성 입력 채널의 전압을, V_{REF}는 기준 전압을 뜻합니다.

10비트 양의 정수인 0(000H)~1023(3FFH) 범위로 총 1,024개의 값으로 변환할 수 있습니다.

ADC 변환 모드는 2가지로, 변환을 한 번만 하고 끝내는 단일 변환 모드(Single Conversion Mode)와 한 번 동작에서 그치지 않고 반복적으로 변환을 하는 프리 러닝 모드(Free Running Mode)가 존재합니다.

단일 입력에 대한 변환 결과는 위에 있는 단극성 입력 A/D 변환식에 따라 도출된 값입니다.

실습에서 사용될 ADC와 관련된 레지스터를 나열하면

1. ADSC : A/D 변환을 시작하는 레지스터 (또한 A/D 변환 완료 시 인터럽트를 발생되도록 할 수 있음.)

2. ADCH, ADCL : A/D 변환의 결과를 저장하는 데이터 레지스터 ADC(ADC Data REgister)
ADC가 ADCH과 ADCL 두 개인 이유는 ATmega128의 ADC는 8비트라 10비트를 맞추지 못하여
ADC를 두 개를 사용해서 8비트x2개 = 16비트로 만들어, 그중 10비트만 사용하고 6개의 비트는 미사용으로 작동합니다.

3. ADMUX : A/D 변환 중 멀티플렉서를 선택하는 레지스터(ADC Multiplexer Selection Register)
A/D 변환기 기준 전압을 선택하고, 결과 값 저장 형식을 하위부터 or 상위부터를 선택할 수 있습니다.
그중 만약 따로 읽을거라면 ADCL을 먼저 읽고 ADCH를 나중에 읽어야 데이터를 손실하지 않고 읽을 수 있습니다.
하지만 a=ADC; 같이 ADC를 통째로 입력하면 편하게 10비트로 ADCL과 ADCH를 모두 읽어올 수 있습니다.

4. ADCSRA : A/D 변환기를 제어 및 상태를 알 수 있는 레지스터 A(ADC Control and Status Register A)

이러한 레지스터를 비롯한 ADC 기능의 실험 원리들을 가지고 AVR studio에서 작성한 코드와 가변저항과 오실로스코프를 통하여 테스트를 진행합니다.

3. 실험기구 및 재료

실험기구 및 재료	수량
ATmega128 마이크로 컨트롤러	1개
컴퓨터	1개
ISP 케이블	1개
프로브 케이블	1개
오실로스코프	1개

표 2) 실험기구 및 재료

4. 실험방법

ADC 기능을 사용하는 테스트를 진행하기 위해 AVR Studio에서 코드를 작성 후 가변저항을 조절하며 오실로스코프를 통해 결괏값을 측정합니다.

예제에 기입된 코드를 작성하여 테스트를 통해 가변저항을 조절해 LED의 불이 순차적으로 켜졌다가 꺼지는 현상과 LCD에 Voltage= 0v~5v가 출력되는 것을 확인했습니다.

후에 구술할 내용은 추가 실습에 관해 작성하겠습니다.

추가실습1) ADC의 선형성 확인 -> 0, 1, 2, 3, 4, 5V를 입력(오실로스코프로 확인)하고 LCD 출력을 그래프로 그리고 비교하여 아래와 같은 Offset error와 Gain error를 조사하시오.

먼저 가변저항과 오실로스코프를 조작하여 원하는 결괏값을 얻을 수 있도록 코드를 작성합니다.

```
#include <avr/io.h>
#include <util/delay.h>
```

I/O와 딜레이 함수를 쓰기 위한 헤더파일을 포함한다고 선언합니다. 이번 실습에선 LCD를 사용하기 때문에 LCD 사용 헤더파일을 추가해야 했지만, 헤더파일이 존재하지 않는 관계로 대신 LCD 함수를 작성합니다.

```
unsigned char temp[3]={'0','.','0'}; // 변환값 표시를 위한 상수값 선언 및 초기화
int Result, i;
```

unsigned char 함수로 temp[3]을 소수 첫째짜리까지만 표시하는 상숫값으로 만듭니다.
그리고 Result와 i라는 변수에 숫자를 집어넣어 사용할 수 있도록 int 함수를 통해 변수 선언을 합니다.

```
void lcd_inst(unsigned char value) // 명령어 입력
```

```
{
        PORTA &= ~0x02; // RS=0 → PA1=0
        _delay_ms(1);
        PORTA |= 0x08; // E=1(& RS=0) → PA3=1
        _delay_ms(1);
        PORTC=value; // Instruction 값을 PORTC로 출력
        _delay_ms(1);
        PORTA &= 0x00; // E=0 & RS=0로 clear
        _delay_ms(1);
}

void lcd_init() // LCD 초기화
{
        _delay_ms(50);
        lcd_inst(0x38);
        _delay_ms(1);
        lcd_inst(0b00001100);
        _delay_ms(1);
        lcd_inst(0x01);
        _delay_ms(1);
        lcd_inst(0b00000110);
        _delay_ms(1);
}

void lcd_ch(unsigned char ch) // 문자 출력
{
        PORTA |= 0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=ch;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}

void lcdout(unsigned char add, unsigned char cc) // 문자 주소
{
        lcd_inst(add | 0x80);
        _delay_ms(1);
```

```
        lcd_ch(cc);
}

void lcd_print(char line, char *text) // 문자열 주소
{
        lcd_inst(line | 0x80);
        while(*text)

        lcd_ch(*text++);
}
```

여기까지와 메인 함수의 DDRC, DDRA를 통해 LCD를 사용하기 위해 설정하는 함수를 작성합니다.

```
int main(void)
{
DDRB=0xff; // LED 출력 설정
DDRC=0xff; // LCD 출력 설정
DDRA=0x0A; // LCD 출력 설정
PORTA=0x00;
lcd_init();// LCD 초기화

lcd_print(0x01,"Voltage=")
_delay_ms(1);
```

DDRB, DDRC, DDRA를 통해 LED와 LED를 출력으로 설정하고, LCD를 초기화시키고, LCD 화면에 "Voltage="라는 문자열을 출력하도록 합니다. 그리고 이제 ADC에 대한 초기 설정을 해줍니다.

```
//--- ADC 초기 설정 ----------------------------------------
   ADMUX=0x00;          // 기준전압 → AREF(외부전압), 우측 정렬 저장,
                        // Single ended input ADC0 채널 선택
   ADCSRA=0x87;         // 0b10000111 ADC enable, Division Factor=128
```

ADMUX를 통해 멀티플렉서를 선택합니다. 기준 전압을 외부 전압으로 사용하며, 우측 정렬 저장을 하게끔 하고, ADC0 채널을 선택하게 합니다. 그리고 ADCSRA를 통해 ADEN=1, ADPS2~0=1로 만들어 A/D 변환기의 동작을 허용합니다.

```
   while(1)
   {
      ADCSRA |=0x40; // AD 변환 시작: ADSC=1
      while((ADCSRA&0x10)==0x00);// AD 변환 완료까지 기다림. ADIF=1 check
```

```
        Result = ADC; // Result라는 변수에 ADC 변환 값 저장
        _delay_ms(1); // 변환 시간 확보
        Result=(Result>>2)*50/255;// X.X로 표시하기위해 변환 결과를
        //0~50으로 Scale
        temp[0]=Result/10 + '0'; // 앞자리 LCD 출력 값 추출
        temp[2]=Result%10 + '0';// 소숫점 뒷자리 LCD 출력 값 추출
        for(i=0; i<3; i++)
        {
            lcdout((0x09+i),temp[i]); //LCD로 출력
            }
    _delay_ms(1);
```

Result 변수를 통해 완료된 ADC 변환 결과를 저장합니다. 저장된 Rsult 변수를 통해 temp에 몫과 나머지를 나누어 대입하여 LCD에 출력하도록 합니다. 변수 i를 통해 temp 문자의 배열을 순차적으로 출력할 수 있도록 합니다.

```
    //----- ADC 값을 이진수로 표현하여 LED로 출력 ------------------------------
    if(Result ==0)PORTB=0xff;
    if(0 <Result && Result <= 7) PORTB=0xfe;
    if(7 <Result && Result <=14) PORTB=0xfc;
    if(14<Result && Result <=21) PORTB=0xf8;
    if(21<Result && Result <=28) PORTB=0xf0;
    if(28<Result && Result <=35) PORTB=0xe0;
    if(35<Result && Result <=42) PORTB=0xc0;
    if(42<Result && Result <=49) PORTB=0x80;
    if(Result ==50) PORTB=0x00;
    }
}
```

마지막으로 Result에 저장된 ADC 변환 결과를 통해 Result=0~50의 값 동안 LED에 출력될 수 있도록 코드를 작성하여 마무리합니다.

이렇게 하여 완성된 코드를 ATmega128 마이크로 컨트롤러 보드에 넣어준 후, 오실로스코프의 화면에서 측정해야 하기 위해 프로브라는 케이블을 연결합니다. 먼저 프로브의 커넥터를 오실로스코프 본체에 연결한 후, 측정의 정확성을 높이기 위해 프로브의 그라운드 클립을 보드 아랫면에 회로 부분에 있는 ATmega128의 접지(Ground)와 연결합니다. 마지막으로 프로브 팁을 보드 윗면을 바라보고 ATmega128 칩의 PF0(Port F의 0번)에 접촉시킵니다.

이상적인 ADC와 다른 실제 ADC를 측정하기 위해 가변저항을 드라이버로 돌려가며 0, 1, 2, 3, 4, 5V를 맞춰서 입력하였을 때, 실제로 오실로스코프에서 측정되는 평균전압을 비교하며 기록하고, 측정된 값을 갖고 그래프를 그려 실습교재에 있는 Offset error와 Gain error를 대조시켜 비교합니다.

추가실습2) ADC 입력으로 PWM 시비율(duty)을 조절해보시오. (LCD 출력)

```
#include <avr/io.h>
#include <util/delay.h>

unsigned char temp[3]={'0','.','0'}; // 변환값 표시를 위한 상수값 선언 및 초기화
int Result, i;

void lcd_inst(unsigned char value) // 명령어 입력
{
        PORTA &= ~0x02; // RS=0 → PA1=0
        _delay_ms(1);
        PORTA |= 0x08; // E=1(& RS=0) → PA3=1
        _delay_ms(1);
        PORTC=value; // Instruction 값을 PORTC로 출력
        _delay_ms(1);
        PORTA &= 0x00; // E=0 & RS=0로 clear
        _delay_ms(1);
}

void lcd_init() // LCD 초기화
{
        _delay_ms(50);
        lcd_inst(0x38);
        _delay_ms(1);
        lcd_inst(0b00001100);
        _delay_ms(1);
        lcd_inst(0x01);
        _delay_ms(1);
        lcd_inst(0b00000110);
        _delay_ms(1);
}

void lcd_ch(unsigned char ch)
{
        PORTA |= 0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=ch;
```

```
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}

void lcdout(unsigned char add, unsigned char cc)
{
        lcd_inst(add | 0x80);
        _delay_ms(1);
        lcd_ch(cc);
}

void lcd_print(char line, char *text)
{
        lcd_inst(line | 0x80);
        while(*text)

        lcd_ch(*text++);
}
```

여기까진 위에 실습 내용과 반복되는 코드라 설명을 생략하였습니다.

```
int main(void)
{
   DDRC=0xff; // LCD 출력 설정
   DDRA=0x0A; // LCD 출력 설정
   PORTA=0x00;
   lcd_init();// LCD 초기화
   TCCR0=0x6f;
```

이번엔 PWM 파형을 오실로스코프에서 측정하기 위해, PWM 레지스터 중 하나인 TCCR0을 통해 timer/counter0과 PWM 모드를 사용한다고 설정합니다.

```
   lcd_print(0x01,"Voltage=");
   _delay_ms(1);

   //--- ADC 초기 설정 ----------------------------------------
   ADMUX=0x00; // 기준전압 → AREF(외부전압), 우측 정렬 저장,
   // Single ended input ADC0 채널 선택
   ADCSRA=0b10000111;// ADC enable, Division Factor=128
```

```
    while(1)
    {
        ADCSRA |=0x40; // AD 변환 시작: ADSC=1
        while((ADCSRA&0x10)==0x00);// AD 변환 완료까지 기다림. ADIF=1 check
        Result = ADC; // Result라는 변수에 ADC 변환 값 저장
        _delay_ms(1); // 변환 시간 확보
        Result=(Result>>2)*50/255;// X.X로 표시하기위해 변환 결과를
        //0~50으로 Scale
        temp[0]=Result/10 + '0'; // 앞자리 LCD 출력 값 추출
        temp[2]=Result%10 + '0';// 소숫점 뒷자리 LCD 출력 값 추출
        for(i=0; i<3; i++)
        {
        lcdout((0x09+i),temp[i]); //LCD로 출력
        }
        _delay_ms(1);

        OCR0=Result*(255/50);

        }
}
```

OCR0을 통해 Result 값을 원래 값으로 되돌려서 듀티 사이클을 출력하도록 합니다. 그리고 코드를 마무리합니다.

위 실습과 같이 프로브의 커넥터를 오실로스코프 본체에 연결한 후, 측정의 정확성을 높이기 위해 프로브의 그라운드 클립을 보드 아랫면에 회로 부분에 있는 ATmega128의 접지(Ground)와 연결한 후, 이번엔 아까와 다른 핀인 OC0 핀에 해당하는 ATmega128 칩의 PB4(Port B의 4번)에 프로브 팁을 접촉합니다.

앞선 실습과 차이가 있는 결괏값을 얻기 위해 입력을 1.3, 1.8, 3.2, 3.7V를 맞춰서 입력하였을 때, 오실로스코프에서 출력되는 듀티비를 측정하여 기록합니다.

5. 측정값

추가실습1) ADC의 선형성 확인 -> 0, 1, 2, 3, 4, 5V를 입력(오실로스코프로 확인)하고 LCD 출력을 그래프로 그리고 비교하여 아래와 같은 Offset error와 Gain error를 조사하시오.

입력	출력
0V	평균 전압 : 3.4mV
1V	평균 전압 : 1.0038V
2V	평균 전압 : 1.9927V
3V	평균 전압 : 2.9129V
4V	평균 전압 : 3.9201V
5V	평균 전압 : 4.8205V

표 3) 입력 전압을 조절하여 나온 출력값과 비교하는 ADC 테스트 측정값

추가실습2) ADC 입력으로 PWM 시비율(duty)을 조절해보시오. (LCD 출력)

입력	출력
1.3V	평균전압 : 1.3451V, 듀티비 : 26.1%
1.8V	평균전압 : 1.7966V, 듀티비 : 36.1%
3.2V	평균전압 : 3.1203V, 듀티비 : 64.0%
3.7V	평균전압 : 3.56V, 듀티비 : 74.3%

표 4) 외부 입력을 통해 duty 조절하는 ADC 기능 테스트 측정값

(실험결과가 한 페이지에 들어있으면 가독성에 좋을 것 같아 다음 페이지로 넘겼습니다.)

6. 실험결과

추가실습1) ADC의 선형성 확인 -> 0, 1, 2, 3, 4, 5V를 입력(오실로스코프로 확인)하고 LCD 출력을 그래프로 그리고 비교하여 아래와 같은 Offset error와 Gain error를 조사하시오.

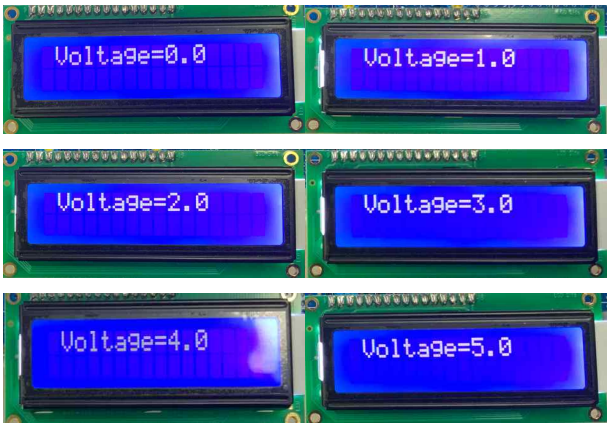


그림 1) 입력 전압



그림 2) 출력 전압

확실히 가변저항을 드라이버로 조절해가면서 넣어준 입력 전압과 실제 오실로스코프에 출력되는 평균전압은 조금씩 차이가 있었다. 입력이 0V일 땐 3.4mV으로 0에 근접하지만 아주 작은 증가량이 있었고, 입력이 1V일 때 또한 1.0038V로 아주 작은 증가량이 있었다. 하지만 이후 입력에 2V~5V를 넣어주었을 때부터는 1.9927V~4.8205V로 입력된 전압보다 출력이 감소하게 되는 모습을 관찰할 수 있었다.

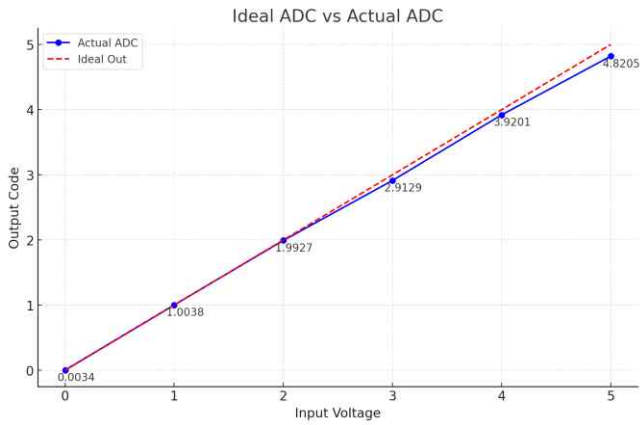


그림 3) 이상적인 ADC와 실제 ADC

실제로 측정한 값을 그래프를 그려서 Offset error와 Gain error를 대조하며 비교한 결과.
이상적인 ADC라 하면 1V를 입력했을 때 1V의 출력이 나와야 하지만, 가변저항을 돌려 입력 전압을 조절하며 얻은 실제 ADC 값은 이상적인 ADC와는 다르게 (입력, 출력) : (0V, 0.0034V), (1V, 1.0038V), (2V, 1.9927V), (3V, 2.9129V), (4V, 3.9201V), (5V, 4.8205V)로 입력값보다 조금 더 높거나 낮은 출력값을 보여줬습니다. 이번 측정값은 0에서 값이 확 벗어나는 Offset error보다는 측정값이 Ideal에서 조금씩 벗어나는 이득 에러(Gain error)를 얻었다고 볼 수 있다.

추가실습2) ADC 입력으로 PWM 시비율(duty)을 조절해보시오. (LCD 출력)

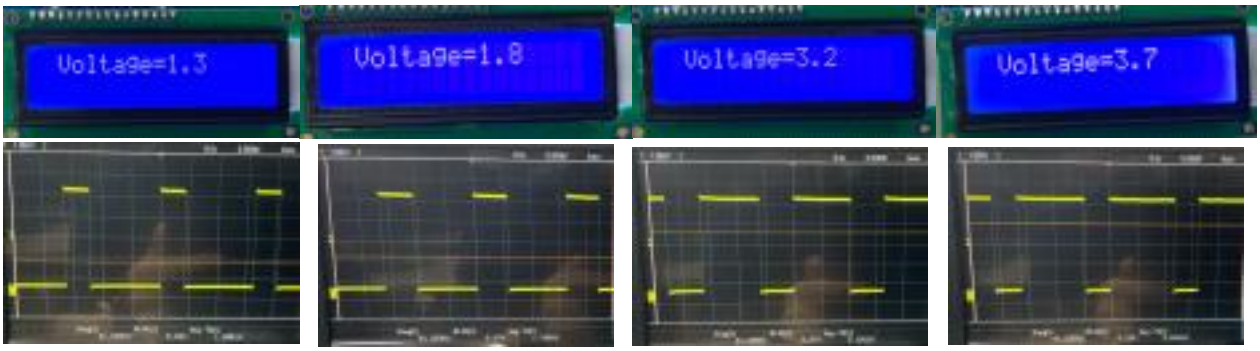


그림 4) 가변저항을 통한 듀티비 조절과 오실로스코프에 출력되는 파형

가변저항을 조절하여 입력된 전압에 따라 듀티비를 조절하는 것이 목표였기 때문에, 입력 전압은 1.3, 1.9, 3.2, 3.7 이렇게 임의의 전압으로 설정하고 듀티비를 측정하였다.

첫 번째 테스트에선 OCR0에 입력되는 Result값이 너무 작아 오실로스코프에 제대로 파형이 출력되지 않았다. 원래 255까지 저장되던 ADC 변환 결과를 LCD 화면에 띄우기 위해 255으로 나누고 50을 곱해주었기 때문에 정상적으로 되지 않았다는 사실을 알고, 다시 255를 통해주고 50을 나눠주어 초기 상태의 ADC 변환결과를 오실로스코프 화면에 정상적으로 띄울 수 있게 되었다.

확실히 입력한 아날로그 값이 낮을수록, 낮은 디지털 출력값을 나타냈다. 낮은 전압에선 1V에 해당하는 라인의 듀티비가 좁고, 0V에 해당하는 듀티비가 크다는 사실을 배웠다.

7. 결과에 대한 논의

아쉬웠던 점은 반복해서 다른 결괏값을 출력해야 하는데 이것을 어떻게 동작시킬지 고민하고 코드를 작성했는데. 프리러닝이란 모드의 존재를 잊고 프리러닝으로 작동시켜야 했던 동작을 단일 모드에서 while 문을 통해 대체했다는 것이다. 주어진 조건에서 활용할 수 있는 기능을 온전히 다 이용하지 못해서 아쉬운 부분이었다. 하지만 사용법을 알고 이제 고쳐야 할 점을 알았으니 아쉬움보단 한 번 더 배우는 과정이라고 생각해서 좋게 생각한다.

좋았던 점은 저번 PWM 기능을 실습할 때는 외부 인터럽트 입력인 스위치를 통해 듀티비를 조절했었는데, 이번엔 가변저항을 드라이버로 돌려가며 미세하게 아날로그 값을 조절하여 그것을 디지털로 변환하여 오실로스코프에서 바로바로 측정할 수 있었던 과정이 흥미로웠다. 스위치보다 섬세한 조절이 가능했고, 마치 영화 속에서 주파수 대역을 잡는 첩보요원이 된 느낌이 들었다.

8. 결론

ATmega128 마이크로 컨트롤러 보드의 ADC 기능을 통해 실제 물리적인 아날로그 입력 값을 디지털 출력값으로 변환하여 직접 눈으로 확인할 수 있었다. 내가 입력한 아날로그값을 알기 때문에 디지털에서 어떻게 출력될지 어느정도 예상이 되었고, 그것을 통해 미세한 조절이 빠르게 가능한 점을 알게 되었다. 다음 실습엔 ATmega128 마이크로컨트롤러 보드로 무엇을 할 수 있을지 기대가 된다.

9. 참고문헌 및 출처

(교재) 김도인 교수님,「실습교재10」, 강원대학교, 전기전자공학과, (2~14p)
(교재) 김도인 교수님 [2024] 마프_Part 10_AD 변환_ver3, "", (2~30p)
(사진) 이동제와 강민수, 실습 중 찍은 사진, 2024-05-30